🧠 Conditional Statements in Shell Scripting

🔹 1. What are Conditional Statements?

Conditional statements allow you to **make decisions in a script**.

📌 In simple terms:
👉 “If a condition is true → do something, else → do something else”

🔹 2. Basic `if` Statement

🔸 Syntax:

```
if [ condition ]
then
 command
fi
```

🔸 Example:

```
a=10

if [ $a -gt 5 ]
then
 echo "a is greater than 5"
fi
```

🔹 3. `if-else` Statement

```
if [ condition ]
then
 command1
else
 command2
```

```
fi
```

- **Example:**

```
read -p "Enter number: " num

if [ $num -gt 0 ]
then
 echo "Positive number"
else
 echo "Negative or zero"
fi
```

---

4. `if-elif-else` (Multiple Conditions)

```
if [ condition1 ]
then
 command1
elif [ condition2 ]
then
 command2
else
 command3
fi
```

- **Example:**

```
read -p "Enter marks: " marks

if [ $marks -ge 90 ]
then
 echo "Grade A"
elif [ $marks -ge 50 ]
then
 echo "Pass"
else
 echo "Fail"
fi
```

---

5. Comparison Operators

📊 Numeric Comparison

Operator	Meaning
`-eq`	Equal
`-ne`	Not equal
`-gt`	Greater than
`-lt`	Less than
`-ge`	Greater or equal
`-le`	Less or equal

🔸 Numeric Comparison Operators with Examples

🔹 1. `-eq` (Equal)

👉 Checks if two numbers are equal

```
a=10
b=10

if [ $a -eq $b ]
then
 echo "a is equal to b"
fi
```

✅ Output:

```
a is equal to b
```

🔹 2. `-ne` (Not Equal)

👉 Checks if two numbers are NOT equal

```
a=10
b=5

if [ $a -ne $b ]
```

```
then
 echo "a is not equal to b"
fi
```

✅ Output:

```
a is not equal to b
```

🔹 3. `-gt` (Greater Than)

👉 Checks if left value is greater

```
a=10
b=5

if [ $a -gt $b ]
then
 echo "a is greater than b"
fi
```

✅ Output:

```
a is greater than b
```

🔹 4. `-lt` (Less Than)

👉 Checks if left value is smaller

```
a=5
b=10

if [ $a -lt $b ]
then
 echo "a is less than b"
fi
```

✅ Output:

```
a is less than b
```

🔹 5. `-ge` (Greater Than or Equal)

👉 Checks if greater OR equal

```
a=10
b=10

if [ $a -ge $b ]
then
 echo "a is greater than or equal to b"
fi
```

✅ Output:

```
a is greater than or equal to b
```

🔹 6. `-le` (Less Than or Equal)

👉 Checks if less OR equal

```
a=5
b=10

if [ $a -le $b ]
then
 echo "a is less than or equal to b"
fi
```

✅ Output:

```
a is less than or equal to b
```

🔹 6. String Comparison

Operator	Meaning
`=`	Equal
`!=`	Not equal
`-z`	Empty string
`-n`	Not empty

🔸 String Comparison Operators with Examples

🔹 1. = (Equal)

👉 Checks if two strings are equal

```
str1="Linux"
str2="Linux"

if [ "$str1" = "$str2" ]
then
 echo "Strings are equal"
fi
```

✅ Output:

```
Strings are equal
```

🔹 2. != (Not Equal)

👉 Checks if two strings are NOT equal

```
str1="Linux"
str2="DevOps"

if [ "$str1" != "$str2" ]
then
 echo "Strings are not equal"
fi
```

✅ Output:

```
Strings are not equal
```

🔹 3. -z (Empty String)

👉 Checks if string is empty

```
name=""

if [ -z "$name" ]
then
```

```
 echo "String is empty"
fi
```

✅ Output:

```
String is empty
```

🔹 4. `-n` (Not Empty String)

👉 Checks if string is NOT empty

```
name="Ashvi"

if [ -n "$name" ]
then
 echo "String is not empty"
fi
```

✅ Output:

```
String is not empty
```

🔹 7. File Test Operators

Operator	Meaning
`-f`	File exists
`-d`	Directory exists
`-r`	Readable
`-w`	Writable
`-x`	Executable
`-s`	File not empty

🔸 File Test Operators with Examples

🔹 1. `-f` (File exists)

👉 Checks if a **regular file exists**

```
file="test.txt"

if [ -f "$file" ]
then
 echo "File exists"
else
 echo "File does not exist"
fi
```

🔹 2. `-d` (Directory exists)

👉 Checks if a **directory exists**

```
dir="myfolder"

if [ -d "$dir" ]
then
 echo "Directory exists"
else
 echo "Directory does not exist"
fi
```

🔹 3. `-r` (Readable)

👉 Checks if file has **read permission**

```
file="test.txt"

if [ -r "$file" ]
then
 echo "File is readable"
else
 echo "File is not readable"
fi
```

🔹 4. `-w` (Writable)

👉 Checks if file has **write permission**

```
file="test.txt"

if [ -w "$file" ]
then
 echo "File is writable"
else
 echo "File is not writable"
fi
```

🔹 5. `-x` (Executable)

👉 Checks if file has **execute permission**

```
file="script.sh"

if [ -x "$file" ]
then
 echo "File is executable"
else
 echo "File is not executable"
fi
```

🔹 6. `-s` (File not empty)

👉 Checks if file exists **and is NOT empty**

```
file="test.txt"

if [ -s "$file" ]
then
 echo "File is not empty"
else
 echo "File is empty or does not exist"
fi
```

🔹 8. Using `[ [ ] ]` (Advanced & Recommended)

👉 Better than `[ ]`

```
if [[ $name == L* ]]
then
```

```
 echo "Starts with L"
fi
```

📌 Advantages:

- Supports pattern matching
- Safer with strings

🔹 9. Logical Operators

Operator	Meaning
`&&`	AND
`||`	OR
`!`	NOT

🔸 Logical Operators in Shell Scripting

🔹 1. `&&` (AND)

👉 Both conditions must be **TRUE**

🔸 Example:

```
a=10
b=5

if [ $a -gt 5 ] && [ $b -lt 10 ]
then
 echo "Both conditions are true"
fi
```

✅ Output:

```
Both conditions are true
```

❌ Example (one false):

```
if [ $a -gt 5 ] && [ $b -gt 10 ]
then
 echo "Won't print"
fi
```

👉 Second condition is false → no output

🔹 2. | | (OR)

👉 At least **one condition must be TRUE**

🔸 Example:

```
a=10
b=5

if [ $a -gt 5 ] || [ $b -gt 10 ]
then
 echo "At least one condition is true"
fi
```

✅ Output:

```
At least one condition is true
```

❌ Example (both false):

```
if [ $a -lt 5 ] || [ $b -gt 10 ]
then
 echo "Won't print"
fi
```

👉 Both conditions false → no output

🔹 3. ! (NOT)

👉 Reverses the condition

🔸 **Example:**

```
a=5

if ! [ $a -gt 10 ]
then
 echo "Condition is false, so NOT makes it true"
fi
```

✅ Output:

```
Condition is false, so NOT makes it true
```

🔹 10. Nested `if`

👉 A **nested if** means:

> Writing an `if` statement **inside another `if`**

📌 Used when you want to check **multiple levels of conditions**

Nested `if` Syntax:

```
if [ condition1 ]
then
 if [ condition2 ]
 then
   command
 fi
fi
```

Nested `if` Example:

```
file="test.txt"

if [ -f "$file" ]
then
if [ -r "$file" ]
then
```

```
echo "File exists and is readable"
else
echo "File exists but not readable"
fi
else
echo "File does not exist"
fi
```

🔹 11. One-Line `if` (Shortcut)

```
[ $a -gt 5 ] && echo "Yes"
```

🔹 12. `case` Statement (Alternative to if-elif)

👉 `case` is used for **multi-way decision making**

📌 In simple terms:
👉 Instead of writing multiple `if-elif`, you use `case` (cleaner & readable)

🔸 Syntax:

```
case variable in
 pattern1)
   command
   ;;
 pattern2)
   command
   ;;
 *)
   default_command
   ;;
esac
```

🔸 Example:

```
#!/bin/bash

echo "Select operation"
echo "a. To print date"
echo "b. To print hostname"
echo "c. To print current directory content"
```

```
echo "d. To check the current location"

read choice

case $choice in
	a)
		echo "Today's date is:"
		date
		echo "Ending"
		;;
	b) hostname;;
	c) ls -l;;
	d) pwd;;
	*) echo "Please provide a valid choice"
esac
```

🔹 13. Exit Status ($?)

👉 Every command returns status:

- `0` → success
- non-zero → failure

```
ls file.txt

if [ $? -eq 0 ]
then
 echo "File exists"
else
 echo "Not found"
fi
```

🔹 14. Common Mistakes

❌ Missing spaces:

```
if [$a -gt 5]
```

✅ Correct:

```
if [ $a -gt 5 ]
```

❌ Not quoting strings:

```
if [ $name = Linux ]
```

✅ Correct:

```
if [ "$name" = "Linux" ]
```

❌ Using `=` for numbers:

```
if [ $a = 10 ]
```

👉 Works but not recommended
✅ Use:

```
if [ $a -eq 10 ]
```

🔹 15. Best Practices

✔ Use `[[ ]]` for conditions
✔ Always quote variables `"${var}"`
✔ Use meaningful conditions
✔ Validate user input

🎯 Summary

✔ `if`, `if-else`, `if-elif` used for decisions
✔ Supports numeric, string, file conditions
✔ `case` is cleaner for multiple choices
✔ `[[ ]]` is safer than `[ ]`